



Отказоустойчивая обработка событий в RabbitMQ

1. Введение

На прошлом занятии мы сделали первый шаг в мир асинхронного взаимодействия. Мы научились отправлять и получать сообщения. Но в реальных, высоконагруженных системах главный вопрос - **«Что произойдет, если что-то пойдет не так?»**. Что если сервис-получатель упадет в момент обработки? Что если сеть моргнет и сообщение не дойдет до брокера? Что если «битое» сообщение заблокирует всю очередь?

Это занятие посвящено построению **надежной и отказоустойчивой** системы обмена сообщениями. Мы перестанем надеяться на лучшее и начнем проектировать систему, которая готова к сбоям. Мы изучим лучшие практики, которые позволяют гарантировать доставку и обработку каждого критически важного события.

Представьте систему интернет-магазина. Сервис Заказов создает заказ и публикует событие OrderCreated. На него подписаны сервисы Оплаты, Склада и Уведомлений.

Что если сервис Оплаты упал?

Что если в событии ошибка, и сервис Склада падает при каждой попытке его обработать?

Практики, которые мы изучим сегодня, позволяют построить систему, где оплата **гарантированно** произойдет, как только сервис поднимется, а сбой на складе не повлияет на работу других систем, и проблемное сообщение будет отложено для анализа инженерами.

Необходимое ПО и настройка среды:

Вам понадобятся все проекты из предыдущего занятия: events-contract, demo-rest, audit-service, а также запущенный контейнер RabbitMQ.

2. Теория

Контракт гарантированной доставки

Надежная доставка — это трехстороннее соглашение:

Producer -> Broker (Издатель -> Брокер)

Издатель должен быть уверен, что брокер получил и сохранил сообщение. Для этого используется механизм **Publisher Confirms**. Издатель ждет от брокера подтверждения.

Broker (Хранение)



Брокер должен сохранить сообщение так, чтобы оно пережило его перезагрузку или сбой. Для этого используются **durable** (долговечные) очереди/обменники и **persistent** (персистентные) сообщения.

Broker -> Consumer (Брокер -> Потребитель)

Брокер должен быть уверен, что потребитель успешно обработал сообщение, прежде чем удалить его из очереди. Для этого используется механизм **Acknowledgements (acks)**.

Ручное подтверждение

По умолчанию Spring Boot использует автоматическое подтверждение (ack). Это значит, что как только брокер отправил сообщение потребителю, он сразу считает его обработанным. **Это опасно!** Если ваш сервис упадет до завершения обработки, сообщение будет потеряно навсегда.

При **ручном подтверждении** (manual ack) брокер ждет от потребителя явной команды:

- **channel.basicAck():** «Я успешно обработал сообщение, можешь его удалять».
- **channel.basicNack(requeue=false):** «Я не смог обработать сообщение, и пытаться снова бесполезно. Не возвращай его в очередь».

Dead Letter Queue (DLQ) - очередь для «мертвых» писем

Что делать с сообщением, которое получило nack (не подтверждено)? Или с сообщением, время жизни которого истекло? Для этого существует **Dead Letter Exchange/Queue**.

Мы настраиваем нашу основную очередь так: «Если сообщение не удалось обработать, отправь его вот в этот dead-letter-exchange». К этому обменнику мы привязываем специальную очередь (my-queue.dlq), где скапливаются все «проблемные» сообщения. Это позволяет:

- **Не блокировать обработку основной очереди.**
- **Инженерам — изучить проблемные сообщения, исправить ошибку и, возможно, переотправить их на обработку.**

Гибкая маршрутизация с Topic Exchange

Topic Exchange позволяет маршрутизировать сообщения на основе шаблонов в routing key.



GraphQL + HAL

- Символ * (звездочка) заменяет ровно одно слово.
- Символ # (решетка) заменяет ноль или более слов.

Пример: routing key = "book.created.hardcover"

- `book.created.*` -> Совпадет
- `book.*.hardcover` -> Совпадет
- `book.#` -> Совпадет
- `*.created.*` -> Совпадет
- `book.created` -> НЕ совпадет (нужно 3 слова)

Это позволяет разным сервисам подписываться на разные срезы событий. Например, сервис статистики может слушать `book.#` (все события о книгах), а сервис уведомлений — только `*.created` (все события о создании чего-либо).

3. Практический разбор

Мы кардинально улучшим нашу существующую систему.

1. **demo-rest** будет публиковать события `book.created` и `book.deleted` с гарантией доставки.
2. **audit-service** будет слушать все события о книгах (`book.*`) и обрабатывать их с ручным подтверждением. Мы настроим для него DLQ.
3. Мы создадим новый сервис, **statistics-service**, который будет слушать только события создания (`book.created`) для пересчета статистики.

Обновление events-contract

1. Создайте новое событие `BookDeletedEvent` в проекте `events-contract`.

```
// events/BookDeletedEvent.java
package edu.rutmiit.demo.events;
import java.io.Serializable;
public record BookDeletedEvent(Long bookId) implements Serializable {}
```

2. Пересоберите контракт

Модификация demo-rest

1. Настраиваем `Publisher Confirms` и персистентность

1. Откройте `application.properties` в `demo-rest`.

... старые настройки ...



Включаем publisher confirms. correlated - для отслеживания, simple - просто true/false

```
spring.rabbitmq.publisher-confirm-type=correlated
```

Включаем publisher returns, чтобы знать, если сообщение не дошло до очереди

```
spring.rabbitmq.publisher-returns=true
```

Шаблон для RabbitTemplate, чтобы все сообщения по умолчанию были персистентными

```
spring.rabbitmq.template.mandatory=true
```

2. Обновляем конфигурацию RabbitMQ

1. Заменяем старую конфигурацию на более явную и надежную.

```
// config/RabbitMQConfig.java
```

```
package edu.rutmiit.demo.demorest.config;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

```
import org.springframework.amqp.core.TopicExchange;
```

```
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
```

```
import org.springframework.amqp.rabbit.core.RabbitTemplate;
```

```
import
```

```
org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class RabbitMQConfig {
```

```
    public static final String EXCHANGE_NAME = "books-exchange";
```

```
    public static final String ROUTING_KEY_BOOK_CREATED =
```

```
"book.created";
```

```
    public static final String ROUTING_KEY_BOOK_DELETED =
```

```
"book.deleted";
```



@Bean

```
public TopicExchange booksExchange() {  
    // durable=true - обменник переживет перезагрузку брокера  
    return new TopicExchange(EXCHANGE_NAME, true, false);  
}
```

@Bean

```
public Jackson2JsonMessageConverter messageConverter() {  
    return new Jackson2JsonMessageConverter(new  
ObjectMapper().findAndRegisterModules());  
}
```

@Bean

```
public RabbitTemplate rabbitTemplate(ConnectionFactory  
connectionFactory, Jackson2JsonMessageConverter messageConverter) {  
    RabbitTemplate rabbitTemplate = new  
RabbitTemplate(connectionFactory);  
    rabbitTemplate.setMessageConverter(messageConverter);  
    // Устанавливаем callback для publisher confirms  
    rabbitTemplate.setConfirmCallback((correlationData, ack,  
cause) -> {  
        if (!ack) {  
            System.out.println("NACK: Message delivery failed! " +  
cause);  
        }  
    });  
    return rabbitTemplate;  
}  
}
```

3. Публикуем событие удаления

1. В BookService добавьте отправку BookDeletedEvent.

// В классе BookService



```
public void deleteBook(Long id) {  
    BookResponse book = findBookById(id); // Убедимся, что книга есть  
    storage.books.remove(id);  
  
    BookDeletedEvent event = new BookDeletedEvent(id);  
    rabbitTemplate.convertAndSend(RabbitMQConfig.EXCHANGE_NAME,  
RabbitMQConfig.ROUTING_KEY_BOOK_DELETED, event);  
}
```

Модификация audit-service

1. Настраиваем ручное подтверждение: В `application.properties` проекта `audit-service`:

```
spring.rabbitmq.listener.simple.acknowledge-mode=manual  
# Отключим встроенные ретраи, чтобы сразу видеть работу DLQ  
spring.rabbitmq.listener.simple.retry.enabled=false
```

2. Обновляем слушателя:

1. Заменяем старый `listener` на новый, который использует ручные `ack`, `DLQ` и `wildcard`-роутинг.

```
// listeners/RabbitMQConfig.java
```

```
@Configuration  
public class RabbitMQConfig {  
  
    @Bean  
    public Jackson2JsonMessageConverter messageConverter() {  
        return new Jackson2JsonMessageConverter(new  
ObjectMapper().findAndRegisterModules());  
    }  
}
```

```
// listeners/BookEventListener.java
```

```
import com.rabbitmq.client.Channel;  
import edu.rutmiit.demo.events.BookCreatedEvent;  
import edu.rutmiit.demo.events.BookDeletedEvent;  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.amqp.rabbit.annotation.*;  
import org.springframework.amqp.support.AmqpHeaders;  
import org.springframework.messaging.handler.annotation.Header;  
import org.springframework.messaging.handler.annotation.Payload;  
import org.springframework.stereotype.Component;
```



| GraphQL + HAL

```
import java.io.IOException;

@Component
@Component
public class BookEventListener {
    private static final Logger log =
        LoggerFactory.getLogger(BookEventListener.class);
    private static final String EXCHANGE_NAME = "books-exchange";
    private static final String QUEUE_NAME = "notification-queue";

    @RabbitListener(
        bindings = @QueueBinding(
            value = @Queue(
                name = QUEUE_NAME,
                durable = "true",
                // если что-то пойдет не так, отправляем в 'dlx-
exchange'
                arguments = {
                    @Argument(name = "x-dead-letter-
exchange", value = "dlx-exchange"),
                    @Argument(name = "x-dead-letter-routing-
key", value = "dlq.notifications")
                },
                exchange = @Exchange(name = EXCHANGE_NAME, type =
"topic", durable = "true"),
                key = "book.created"
            )
        )
        // Используем @Payload для явного указания параметра сообщения
        public void handleBookCreatedEvent(@Payload BookCreatedEvent event,
            Channel channel,
            @Header(AmqpHeaders.DELIVERY_TAG) long
            deliveryTag) throws IOException {
            try {
                log.info("Received BookCreatedEvent: {}", event);
                if (event.title() != null &&
                    event.title().equalsIgnoreCase("CRASH")) {
                    throw new RuntimeException("Simulating processing error for
DLQ test");
                }
                // Логика отправки уведомления...
                log.info("Notification sent for new book '{}'", event.title());
                // Отправляем подтверждение брокеру
                channel.basicAck(deliveryTag, false);
            } catch (Exception e) {
                log.error("Failed to process event: {}. Sending to DLQ.", event,
                    e);
                // Отправляем nack и НЕ просим вернуть в очередь (requeue=false)
                channel.basicNack(deliveryTag, false, false);
            }
        }

    @RabbitListener(
        bindings = @QueueBinding(
            value = @Queue(
                name = QUEUE_NAME,
                durable = "true",
```



```
arguments = {
    @Argument(name = "x-dead-letter-
exchange", value = "dlx-exchange"),
    @Argument(name = "x-dead-letter-routing-
key", value = "dlq.notifications")
}),
exchange = @Exchange(name = EXCHANGE_NAME, type =
"topic", durable = "true"),
key = "book.deleted"
)
)
public void handleBookDeletedEvent(@Payload BookDeletedEvent event,
Channel channel,
@Header(AmqpHeaders.DELIVERY_TAG) long
deliveryTag) throws IOException {
    try {
        log.info("Received BookDeletedEvent: {}", event);
        // Логика отмены уведомлений...
        log.info("Notifications cancelled for deleted bookId {}",
event.bookId());
        channel.basicAck(deliveryTag, false);
    } catch (Exception e) {
        log.error("Failed to process event: {}. Sending to DLQ.", event,
e);
        channel.basicNack(deliveryTag, false, false);
    }
}
}
```

3. Добавляем слушателя для DLQ (опционально, но полезно для отладки):

```
// В том же классе BookEventListener
@RabbitListener(
    bindings = @QueueBinding(
        value = @Queue(name = "notification-queue.dlq", durable =
"true"),
        exchange = @Exchange(name = "dlx-exchange", type = "topic",
durable = "true"),
        key = "dlq.notifications"
    )
)
public void handleDlqMessages(Object failedMessage) {
    log.error("!!! Received message in DLQ: {}", failedMessage);
    // Здесь может быть логика оповещения администраторов
}
```

Запуск и проверка

1. Запустите audit-service, а затем demo-rest.
2. Создайте книгу через POST /api/books с любым названием. В логах audit-service вы увидите сообщение о получении события и отправке уведомления.



GraphQL + HAL

3. Создайте книгу с названием CRASH. В логах audit-service вы увидите ошибку, а затем сообщение от handleDlqMessages о том, что сообщение попало в DLQ!
4. Удалите книгу через DELETE /api/books/{id}. В логах audit-service вы увидите, что он поймал и это событие, потому что его routing key book.deleted совпадает с шаблоном book.*.
5. Зайдите на <http://localhost:15672>. Во вкладке "Queues" вы увидите все созданные очереди, включая notification-queue.dlq.

4. Задания для самостоятельной работы

- Давайте попрактикуемся в создании подписчика с другим шаблоном маршрутизации
- Задача:
 - Создайте новый Spring Boot проект statistics-service.
 - Подключите events-contract, настройте RabbitMQ с manual ack.
 - Создайте слушателя, который будет подписываться с ключом book.created (не book.*!). Ему не интересны удаления.
 - Логика должна быть простой: хранить в Map<Long, String> ID книги и ее название и выводить в лог общее количество книг после каждого нового события.
- Реализуем критически важную практику — защиту от дубликатов.
- Задача:
 - В notification-service создайте private final Set<Long> processedBookCreations = ConcurrentHashMap.newKeySet(); (потокбезопасная версия).
 - В методе handleBookCreatedEvent, перед обработкой BookCreatedEvent, добавьте проверку: if (!processedBookCreations.add(event.bookId())) { log.warn("Duplicate event received for bookId: {}", event.bookId()); channel.basicAck(deliveryTag, false); return; }.
 - Временно закомментируйте channel.basicAck() в основной обработке, перезапустите сервис и отправьте событие. Сообщение не будет подтверждено и вернется снова при перезапуске сервиса. Вы должны увидеть в логге предупреждение о дубликате.



5. Дополнительные материалы

Настройки `application.properties` (шпаргалка):

- `spring.rabbitmq.publisher-confirm-type=correlated`
 - Включить подтверждения от брокера для издателя
- `spring.rabbitmq.listener.simple.acknowledge-mode=manual`
 - Включить ручное подтверждение для потребителя
- `spring.rabbitmq.listener.simple.retry.enabled=false`
 - Отключить встроенные повторные попытки Spring (полезно для отладки DLQ)

Проектируйте потребителей идемпотентными! В распределенных системах нет гарантии «доставки ровно один раз» (exactly-once delivery). Гарантия — **«хотя бы один раз»** (at-least-once). Это означает, что ваш сервис **МОЖЕТ** получить одно и то же сообщение несколько раз (например, если он упал после обработки, но до отправки ack). Ваша бизнес-логика должна быть готова к этому и не создавать дубликаты заказов, не отправлять одно и то же письмо дважды и т.д.